Tema Proiect: TicketFlow

Sistem Distribuit de Management al Evenimentelor si Emitere bilete

Student: Bolfă Robert-Ion **Grupa: 344 C3**

1. Descriere Generala

TicketFlow este o platforma web distributia destinata gestionarii evenimentelor si achizitionarii de bilete online. Aplicatia este conceputa pentru a gestiona un volum mare de cereri simultane (high concurrency), o problema frecventa in sistemele de ticketing (ex: lansarea biletelor la un concert popular).

Sistemul Permite administratorilor sa publice evenimente, iar utilizatorilor sa rezerve si sa cumpere bilete. Platforma abordeaza doua probleme critice ale sistemelor distribuite:

1. Race conditions: Prevenirea vanzarii aceluiasi loc catre doua persoane diferite simultan, folosind un mechanism de distributed locking (rezervare temporara).
2. Latenta: Decuplarea generarii biletelor (PDF) de interactiunea utilizatorului, folosind procesare asincrona.

2. Arhitectura sistemului

Proiectul este structurat ca un system distribuit modular, urmand arhitectura de microservicii. Intreaga solutie va fi livrata sub forma unui stack Docker Swarm, unde fiecare componenta este definite ca un serviciu separate in fisierul de configurare .yml.

Detalii de infrastructura si Securitate:

- Interconectare (Service discovery) : Comunicarea dintre servicii se va realiza exclusive prin numele de DNS interne generate de Docker. Adresele serviciilor (ex: postgres-db, rabbitmq-broker) nu vor fi hardcodate, ci vor fi injectate dynamic in containere prin variabile de mediu.
- Segregarea retelelor (Network isolation): Pentru a maximiza securitatea, serviciile vor fi isolate in retele overlay distincte, conform principiului least privilete:
  - ❾ Public_net: Expune doar API Gateway-ul catre exterior)traffic utilizatori).
  - ❾ App_net: Permite comunicarea intre API Gateway, Keycloak si RabbitMQ.

- Data_net: Retea private izolata, accesibila doar serviciilor care necesita persistenta (Workers <-> DB, API<->REDIS/D), fara access direct din exterior.

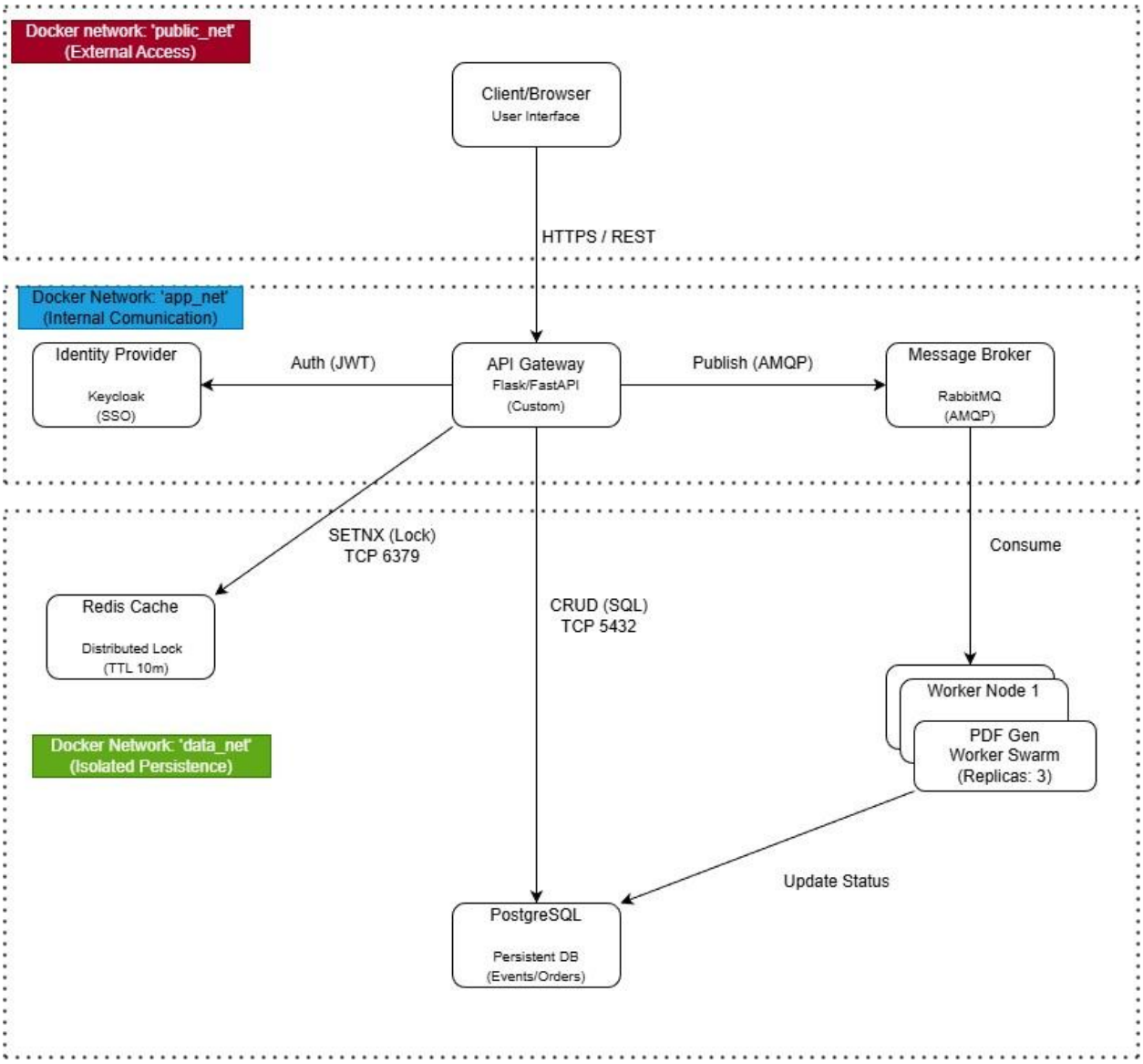
Componentele Sistemului(servicii):

1. **Gateway API & Frontend (Backend Principal)**
   - **Rol:** Punctul central de intrare. Gestioneaza logica de business, verifica disponibilitatea biletelor in timp real si coordoneaza tranzactiile.
   - **Tehnologii:** Python (Flask/FastAPI), SQLAlchemy. **Tip:** Componenta dezvoltata (Custom).
2. **Identity Provider (IDP)**
   - **Rol:** Asigura autentificarea centralizata (SSO) si managementul rolurilor (Admin vs User).
   - **Tehnologii:** Keycloak (imagine oficiala). **Tip:** Componenta Open-Source.
3. **In-Memory Data Store (Cache & Locking)**
   - **Rol:** Gestioneaza starea volatile a rezervarilor. Este folosit pentru a bloca temporar un bilet pentru un utilizator (timp de 10 minute) inainte de plata, asigurand consistenta datelor in scenario cu multi utilizatori concurenti.
   - **Tehnologii:** Redis.
   - **Tip:** Componenta Open-Source.
4. **Database Server**
   - **Rol:** Stocarea persistenta a datelor (evenimente, comenzi finalizate, utilizatori).
   - **Tehnologii:** PostgreSQL.
   - **Tip:** Componenta Open-Source.
5. **Message Broker**
   - **Rol:** Coada de mesaje pentru comunicarea asincrona intre API si procesatorii de bilete.
   - **Tehnologii:** RabbitMQ.
   - **Tip:** Componenta Open-Source.
6. **PDF Processor Workers (scalabil)**
   - **Rol:** Serviciu replicat care preia comenzile confirmate din coada, genereaza biletele in format PDF si le trimite/salveaza.
   - **Scalabilitate:** Acest serviciu va fi definit cu “replicas : N” (minim 3 instante) in configurarea Swarm pentru procesare paralela.
   - **Tehnologii:** Python Script.

- **Tip:** Componenta dezvoltata (Custom).

7. Email Testing Service (SMTP Mock)

-> Rol: Simuleaza un server de email real pentru mediul de dezvoltare. Intercepteaza emailurile trimise de *PDF Workers* (continand biletele) si le expune intr-o interfata web locala.



3. **Functionalitati si Roluri**

Sistemul implementeaza un mechanism de control al accesului bazat pe roluri (RBAC), gestionat prin Keycloak:

A. **Rol: Vizitator (Neautentificat)**

- Vizualizare lista evenimente publice si detalii generale.
- Autentificare/Inregistrare (redirectionare catre Keylcloak).

B. **Rol: Utilizator Standard (Authenticated User)**

- **Rezervare Temporara (Checkout):** La initierea comenzii, un bilet este 'blocat' timp de 10 minute in sistem (Redis). Daca plata nu se efectueaza in acest interval, biletul devine din nou disponibil automat pentru alti clienti (mechanism TTL).
- **Achizitie:** Finalizarea platii transforma rezervarea temporara intr-o comanda permanenta salvata in baza de date.
- **Vizualizare istoric:** Acces la statusul comenzilor (Pending, Processing, Completed).
- **Descarcare Bilet:** Acces la biletul PDF generat asincron

C. **Rol: Administrator (Organizator)**

- **Management Evenimente:** CRUD (Creare, Citire, Actualizare, Stergere) pentru evenimente.
- **Monitorizare:** Vizualizare lista completa de comenzi din sistem.

4. **Functionalitati Avansate (Sisteme Distribuite)**

Penntru cerintele de complexitate , proiectul include:

1. **Managementul Concurentei (Distributed Locking cu Redis):**
   - Implementarea unui mecanism de blocare a resurselor pe durata limitata (Time-To-Live). Aceasta rezolva problema de double-booking fara a bloca baza de date principala, asigurand performanta ridicata.
2. **Procesare Asincrona (Message Queues):**
   - Utilizarea RabbitMQ pentru a procesa generarea documentelor PDF in fundal, Aceasta permite interfetei web sa raspunda instantaneu utilizatorului ("Comanda a fost preluata"), chiar daca generarea efectiva a biletului dureaza, crescand astfel scalabilitatea interfetei.
3. **Scalare orizontala (Replicare):**
   - Componenta de procesare a biletelor (PDF Workers) var ula in multiple containere simultan, demostrand capacitatea sistemului de a scala orizontal in functie de incarcare.